

MediClaim Orchestrator

Our Tech Stack, Explained Simply

What each tool is, what it does in our project, and why we chose it

Multi-Agent Cashless and Reimbursement Claim Processing

October 2026

The office picture

The easiest way to remember the stack: our system is like an office.

Role in the office	Tool
The clerks who read and write	CrewAI agents
The brain the clerks think with	Ollama (gemma4)
The manager who starts jobs on time	n8n
The rule book that decides everything	FastAPI services and the calc engine
The cupboards where things are kept	Postgres, MinIO, Redis, Qdrant
The security guards	ClamAV, Keycloak, Presidio, HMAC
The building everything sits in	Docker

1. The AI part (the thinking)

Tech	What it is	What it does for us	Why we chose it
CrewAI	A Python framework for teams of AI agents	Runs our 11 agents. Hospital: map bill lines, read the policy, sort queries, write replies. Insurer: identity, authenticity, coverage, bill mapping, summary	The guidelines ask for a multi-agent framework. Earlier the 'agents' were plain Python files. CrewAI gives clear roles, step order (Flows) and answer checks (Guardrails)
Ollama	Runs AI models on your own laptop	Serves the AI model to the agents	Patient data never leaves the laptop. Free, no API key, works offline
gemma4	The AI model	The agents think with it: read bills, policies and queries, write drafts	Good quality and small enough for a laptop
nomic-embed-text	A small model that turns text into numbers	Makes policy documents searchable by meaning	Small, fast, local, made for search
Qdrant + RAG service	A search database. RAG = look it up first, then answer	Finds the exact policy paragraph so the agent quotes real text	Stops the AI from making things up. Hospital and insurer have separate collections
Presidio	A tool that finds personal details in text	Hides names, phone and ID numbers before text goes to the AI	Privacy. Health data is sensitive under India's DPDP Act

Why AI only in some places?

AI is great at reading messy text and writing, but it can get numbers wrong and can invent things. So AI reads and writes, and plain code decides and calculates.

2. The workflow part (the manager)

Tech	What it is	What it does for us	Why we chose it
n8n	A visual workflow tool: boxes joined by arrows	Runs 13 hospital and 19 insurer workflows. Starts jobs when something happens, runs timers (every 5 min, 2 am nightly), waits, retries, reminds	Claims take hours or days. n8n is made for waiting, retrying and reminding. You can see the process as a picture. Open source and free

Why both n8n and CrewAI?

CrewAI is the thinking inside one step ('write this reply'). n8n is everything around it ('when do we start, how long do we wait, what if it fails'). Golden rule: n8n only says WHEN; the API decides WHAT.

3. The backend part (the rule book)

Tech	What it is	What it does for us	Why we chose it
Python 3.12	Our main programming language	All the server-side code	AI tools (CrewAI, Presidio, OCR) work best in Python
FastAPI	A framework for web APIs	All 9 services: hospital API, insurer API, both crews, calc engine, doc-pipeline, vision, RAG, bank simulator	Fast, simple, checks every request, writes its own API docs
Pydantic	A data checker	Checks every request, response and agent answer	Bad data is stopped at the door; broken agent output is caught at once
SQLAlchemy + Alembic	Talks to the database; tracks database changes	Saves cases, claims, documents, logs. Upgrades the database step by step (hospital at 0023)	Anyone can build the exact same database with one command
Calc engine (our code)	Plain Python with exact decimals	Works out the payable amount: room rent limit, co-pay, sub-limits, deductions	Money must be exact and explainable, so no AI touches the money

4. The storage part (the cupboards)

Tech	What it is	What it does for us	Why we chose it
PostgreSQL 16	A trusted open-source database	Two databases, one per side. All records and the audit chain	Reliable, free, user roles so each service gets only the access it needs
Redis 7	A super-fast in-memory store	Remembers handled messages (nothing happens twice), rate limits, live screen updates	Very fast for small, short-lived data
MinIO	A file store like Amazon S3, on our laptop	Stores all uploaded scans and PDFs, encrypted	Same interface as S3, so we could move to the cloud later

5. The safety part (the security guards)

Tech	What it is	What it does for us	Why we chose it
ClamAV	A free antivirus	Scans every uploaded file first	Files come from outside; a virus must never get in
Keycloak	A login and roles server	Login for desk, approver, admin; tokens for services	Industry standard. Writing your own login is risky
HMAC signatures	A digital signature with a secret key	Every message between hospital and insurer is signed	Nobody can fake or change a claim on the way
Hash-chained audit log	Each record linked to the one before, like a blockchain	Records every step on both sides	If anyone secretly edits an old record, the chain breaks

6. The document part (the readers)

Tech	What it is	What it does for us	Why we chose it
Poppler	A PDF tool	Turns PDF pages into images	Needed before OCR can read them
Tesseract	OCR: reads text from images	Reads scanned bills and reports	Free, proven, runs locally
OpenCV	An image processing library	Checks blur and looks for the hospital stamp	Catches bad scans early, before they cause queries

7. The screen part

Tech	What it is	What it does for us	Why we chose it
Next.js + React + TypeScript	A framework for web pages	Hospital desk and insurer reviewer screens: checklist, claim, estimate card, queries, decisions	Modern, widely used; TypeScript catches mistakes before running

8. The 'make it run' part

Tech	What it is	What it does for us	Why we chose it
Docker + Compose	Runs software in sealed boxes; Compose starts many at once	Starts Postgres, Redis, MinIO, ClamAV, Keycloak, Qdrant, n8n	Works the same on every computer, no manual installs
uv	A very fast Python installer	Installs all our Python projects at once	Much faster than pip, versions locked
Make	Short named commands	make demo, make demo-down, make crew-demo	One command runs the whole project
pytest, ruff, mypy, Playwright	Testing, style, type checks, browser tests	Hundreds of automatic tests plus screen click-throughs	We know it works, and know when something breaks

If sir asks: why this stack overall?

Say this

"We picked tools that are free, open source and run locally, because health data is private and we didn't want any cost per call or patient data leaving the machine. We used AI only where it's really needed, for reading and writing, through CrewAI as the guidelines require. Everything with rules or money is plain, tested code. n8n manages the long-running process, Docker makes it run the same anywhere, and the whole thing starts with one command."

Quick 'why not' answers

- **Why not OpenAI or ChatGPT?** Patient data would leave the machine, it costs per call, and it needs internet.
- **Why not LangChain?** CrewAI is made for role-based agent teams, with Flows and Guardrails built in, and is simpler to read.

- **Why not cron jobs instead of n8n?** You can't see them as a process, and retries, waits and alerts would all be hand-written.
- **Why not let the AI calculate?** AI can make small number mistakes, and with money one wrong rupee is a real problem.